SOFTWARE ENGINEERING DOCUMENTATION

ODYSSEY

An Expandable, AI-Supported Learning Platform

System Analysis, Requirements, Feasibility, and Design

Chapters 1 to 5

Prepared 25 October 2026

TABLE OF CONTENTS

Chapter 1 INTRODUCTION 1
1.1 Background of the Study 1
1.2 Statement of the Problem 2
1.3 Objectives of the Study 2
1.4 Significance of the Study 3
1.5 Scope and Limitations 3
1.6 Definition of Terms 4
Chapter 2 REVIEW OF RELATED LITERATURE 5
2.1 Interactive Simulations in Computing Education 5
2.2 Multimedia Learning and Visual Explanations 5
2.3 AI-Supported Learning 6
2.4 Gamification and Learner Engagement 6
2.5 Web-Based and Mobile Learning Access 6
2.6 Synthesis and Research Gap 7
Chapter 3 THEORETICAL AND CONCEPTUAL FRAMEWORK 8
3.1 Theoretical Framework 8
3.2 Conceptual Framework 8
3.3 System Specification 10
3.3.1 System Architecture 10
3.3.2 Deployment Topology 11
3.3.3 Use Case Model 12
3.3.4 Learner Study Activity 13
3.3.5 Protected Progress Request Sequence 14
3.3.6 Conceptual Data Model 15
3.3.7 Development Environment and Tools 16
Chapter 4 METHODOLOGY 17
4.1 Research and Development Approach 17
4.2 Agile Development Method 17
4.3 System Requirements 18
4.4 Feasibility Analysis 20
4.5 Verification and Evaluation Plan 22
Chapter 5 RESULTS AND DISCUSSION 23
5.1 Learner Experience and Study Tools 23
5.2 Pengu Study Aids and Feedback 25
5.3 Interactive Operating-Systems Practice 29
5.4 Classroom and Instructor Workflows 30
5.5 Engagement and Administrative Tools 33
5.6 Discussion and Evidence Limitations 37

References 38

LIST OF FIGURES

Figure 1. Input-Process-Output Model of Odyssey 9
Figure 2. Odyssey System Architecture 10
Figure 3. Odyssey Deployment Topology 11
Figure 4. Main Use Case Diagram for Odyssey 12
Figure 5. Learner Study Activity Diagram 13
Figure 6. Lesson Completion Request Sequence Diagram 14
Figure 7. Conceptual Entity Relationship Diagram 15
Figure 8. Agile Software Development Cycle 17
Figure 9. Odyssey home screen and study options 24
Figure 10. Module route and learning progress 25
Figure 11. PDF source and Pengu study options 26
Figure 12. Flashcard question state 27
Figure 13. Flashcard answer state 28
Figure 14. Quick Check question and feedback 29
Figure 15. System Call Tracer interface 30
Figure 16. My Classrooms and class-joining interface 31
Figure 17. Classroom modules and displayed progress 32
Figure 18. Instructor application opening screen 33
Figure 19. Odyssey learner XP leaderboard 34
Figure 20. Administrator overview statistics 35
Figure 21. Administrator user management 36
Figure 22. Administrator classroom management 37

LIST OF TABLES

Table 1. Development Technology Stack 16

Table 2. Functional Requirements 18

Table 3. Non-Functional Requirements 19

Table 4. Economic Feasibility Assessment 20

Table 5. Planned Verification 22

Chapter 1

INTRODUCTION

1.1 Background of the Study

Digital learning platforms can bring lesson content, practice, feedback, and classroom activities into one accessible environment. In computer science education, however, topics such as process scheduling, memory management, concurrency, and file systems describe changing states and interactions that are difficult to observe through static text alone. Odyssey addresses this instructional challenge with guided lessons and browser-based simulations that let learners select inputs and inspect step-by-step behavior.

Odyssey is an expandable learning platform delivered as a cross-platform web application and installable Progressive Web App. Its current Explore Lessons collection focuses on operating-systems concepts, including processes, threads, memory, file systems, system calls, and scheduling. The learning experience also includes quizzes, flashcards, PDF study sessions, progress indicators, profiles, XP, badges, and leaderboards. Pengu provides general-purpose AI-supported tutoring across subjects, while separate AI workflows generate quizzes and flashcards from supported materials.

The platform also supports instructors who organize classrooms, publish learning materials, create quizzes, and review submissions. Administrators have workflows for account and role management, instructor applications, classrooms, statistics, and audit activity. The web client is implemented with HTML, CSS, and JavaScript. A service worker caches the application shell and selected static assets; authentication, database, API, and AI features rely on network connectivity. A FastAPI service connects authenticated workflows and AI requests with Supabase Auth and PostgreSQL and configured external AI services [1]-[4].

The current implementation is an educational platform, not a production operating-system emulator. The simulations visualize selected concepts and algorithm behavior; they do not execute guest operating systems or represent real hardware workloads. Odyssey therefore provides a structured way to study and experiment with operating-systems concepts while leaving room for additional subject collections as content is developed.

1.2 Statement of the Problem

Learners studying operating-systems concepts may have difficulty connecting abstract explanations to changing process, memory, scheduling, and concurrency states. Static study

materials provide limited opportunity to vary inputs and observe the sequence of algorithm decisions. Learning activities, study aids, progress records, and classroom tasks may also be spread across separate tools, making it harder to maintain a connected study workflow.

Odyssey is designed to address this instructional and system-design problem through one expandable platform that combines guided lessons, interactive simulations, study tools, learner progress, AI-supported assistance, and classroom functions. The documentation must describe the implemented scope and its constraints accurately. Repository evidence does not establish that all deployment environments are configured, that the platform has passed formal user acceptance, or that it improves learning outcomes.

1.3 Objectives of the Study

The general objective is to document the analysis and design of Odyssey as an accessible, installable learning platform that can grow across subjects and currently supports operating-systems instruction through guided content, interactive practice, knowledge checks, and progress feedback.

The specific objectives are to:

- provide the current operating-systems lessons and labs while supporting future additions to the Explore Lessons catalog;
- provide interactive simulations in which learners can configure or select scenarios and inspect algorithm behavior;
- support learner accounts, profiles, progress indicators, XP, badges, and leaderboard features;
- provide a general-purpose AI chatbot and AI quiz and flashcard generation from supported study materials;
- support instructor workflows for classrooms, learning materials, quizzes, and learner submissions; and
- provide administrator workflows for accounts, roles, applications, classrooms, statistics, and activity oversight.

1.4 Significance of the Study

Learners. Odyssey brings lessons, simulations, quizzes, flashcards, PDF study sessions, progress summaries, and AI-supported assistance into a shared learning environment. Interactive labs can help learners inspect how selected inputs affect modeled operating-systems behavior.

Instructors. Classroom tools support organizing learners and learning materials, publishing modules, preparing quizzes, and reviewing submissions. The value of these workflows depends on appropriate role assignment and institutional use.

Administrators and adopters. Role-aware administration provides account, role, application, classroom, statistics, and audit workflows. Adoption still requires operational ownership, privacy practices, provider configuration, and deployment monitoring.

Future developers and researchers. The modular client, FastAPI service, Supabase integration, and conceptual models provide a reference for extending the platform and planning formal evaluation. Further research is needed to assess usability, accessibility, reliability, learning outcomes, and operating costs.

1.5 Scope and Limitations

The documented system includes a cross-platform web client and installable PWA; operating-systems lessons and browser-side simulations; quizzes, flashcards, and PDF study workflows; learner profiles and progress features; classroom and administration workflows; a FastAPI backend; Supabase Auth and PostgreSQL integration; configured AI study endpoints; and deployment definitions targeting Vercel and Render [1]-[5].

The built-in Explore Lessons catalog currently focuses on operating systems. Additional subjects require lesson content and supporting activities before they can appear in the catalog. Pengu is intended to answer questions across subjects, but its availability and behavior depend on backend configuration and an external AI provider. The service worker caches the app shell and selected static assets; live API, authentication, database, and AI operations need a network connection. Some preferences, quiz history, simulator state, and viewed-lesson markers are held in browser local storage and may not synchronize across devices.

This documentation is based on the supplied project repository and existing system documentation. It does not claim that a production environment is active or correctly configured. No measured response-time, concurrency, accessibility-conformance, uptime, cost, user-acceptance, security-certification, or learning-outcome evidence was supplied. Simulations are educational models rather than real operating-system execution. AI-generated content may be inaccurate and requires learner judgment and review.

1.6 Definition of Terms

Administrator. An authorized user with access to supported account, role, application, classroom, statistics, and audit workflows.

Artificial intelligence (AI) study support. Configured chatbot and content-generation workflows that use an external AI service through the backend.

Classroom. A platform workspace through which an instructor organizes members, modules, and quizzes.

Learner. A user who studies lessons, runs simulations, completes quizzes, and reviews available progress information.

Progressive Web App (PWA). A web application that can be installed on supported devices and uses browser capabilities such as a web manifest and service worker.

Row Level Security (RLS). Database policies that control which rows an authenticated role may read or change.

Simulation. A browser-based educational visualization of a selected operating-systems process or algorithm, not a live operating-system environment.

Supabase. The configured backend platform used for authentication and PostgreSQL data services.

Instructor. An authorized user who manages classroom membership, learning materials, quizzes, and related submissions.

Explore Lessons. The learner-facing catalog of modules and topics; its current built-in subject collection is operating systems.

Chapter 2

REVIEW OF RELATED LITERATURE

2.1 Interactive Simulations in Computing Education

Computer simulations can make a conceptual model available for learner experimentation. In their review of discovery learning with simulations, de Jong and van Joolingen [6] describe how learners can infer model behavior by changing conditions and observing outcomes. They also identify difficulties associated with unguided discovery and discuss instructional support that can help learners use simulations productively. This finding supports coupling a simulation with explanations and structured learning activities rather than presenting an unexplained sandbox.

The principle is relevant to operating-systems topics because many concepts involve state transitions, ordered decisions, and resource constraints. A scheduling lab, for example, can make an algorithm's selection order and waiting times visible when the learner changes a workload. A process or memory lab can expose intermediate states that are otherwise described symbolically. Odyssey's simulations follow this instructional pattern by allowing learners to select inputs and inspect traces, while lesson content supplies context. The models remain simplifications and should be interpreted as teaching aids rather than measurements of a real computer.

2.2 Multimedia Learning and Visual Explanations

Mayer's cognitive theory of multimedia learning describes how learners process verbal and pictorial representations through limited-capacity channels and integrate them with prior knowledge [7]. The theory provides a basis for pairing concise explanations with visual representations when a concept depends on spatial relationships, sequences, or changes over time. Interactive visualizations can extend that pairing by allowing a learner to manipulate selected inputs and observe modeled consequences.

For Odyssey, visual traces and diagrams can help explain process states, scheduling order, memory placement, and thread interactions. These visuals need clear labels and explanations so that learners can connect each displayed state with the underlying concept. A visual display alone does not establish learning effectiveness, and cluttered or unsupported animation can distract from the instructional objective. This study treats visual simulations as a design feature whose usability and educational value require later evaluation.

2.3 AI-Supported Learning

AI-based tutoring can provide responsive explanations and practice prompts, but evidence about outcomes depends on how the tutor is designed and evaluated. Kestin et al. [8] reported a randomized controlled study in an introductory university physics course in which a research-designed AI tutor was compared with active classroom instruction. The study provides evidence that an AI tutor can support learning in a particular course when it is structured around the instructional material. Its setting and tutor design differ from Odyssey, so the results cannot be transferred directly to Odyssey or to other subjects.

Odyssey includes Pengu as a general-purpose chatbot and separate workflows for generating quizzes and flashcards from supported study materials. The platform routes AI requests through the backend so provider credentials remain outside the browser. Generated explanations and study items can be useful prompts for review, but they may be incomplete or incorrect. The system therefore should be evaluated for answer quality, relevance, safety, response time, and learner understanding. The current project records do not provide those evaluation results.

2.4 Gamification and Learner Engagement

Gamification applies selected game elements, such as points, badges, or progress displays, to a non-game activity. Sailer and Homner [9] synthesized experimental evidence about gamification in education and found that effects depend on the specific intervention and learning outcome. Their review cautions against treating game mechanics as a guarantee of improved learning. Progress indicators may make completion visible and support goal setting, but they should remain connected to meaningful learning activities.

Odyssey includes XP, streaks, badges, profiles, statistics, and leaderboards as engagement and progress features. These mechanisms can communicate activity and completion, while quizzes, lessons, and simulations provide the underlying learning tasks. Whether these features improve persistence or learning for Odyssey users is an empirical question and is not established by the code inventory or system design.

2.5 Web-Based and Mobile Learning Access

Browser-based delivery can reduce the need for learners to install a platform-specific application, while PWA capabilities can support installation and caching of selected static resources on supported devices. Odyssey's manifest and service worker are intended to provide

this app-like access and cache the application shell. The cached shell does not make hosted authentication, database operations, or AI workflows available offline; those features depend on configured services and connectivity [5].

Cross-device access also depends on responsive interfaces, browser compatibility, accessible interaction, and reliable server configuration. The repository includes desktop and mobile-oriented client pages, but source inspection alone does not establish conformance with an accessibility standard or consistent behavior on every device. These concerns inform the platform's non-functional requirements and future verification plan.

2.6 Synthesis and Research Gap

The reviewed literature supports several design choices in Odyssey: guided simulations can make conceptual models available for experimentation; verbal explanations and visual representations can work together; AI tutoring can support learning when it is designed around instruction; and gamification may support engagement when connected to meaningful tasks. PWA delivery can broaden access to the web client, although network-dependent services remain subject to connectivity and deployment conditions.

The cited work does not evaluate Odyssey, its operating-systems lessons, its specific simulations, its general-purpose Pengu chatbot, or its classroom workflows. The project documentation also does not include a controlled learning study, formal usability evaluation, accessibility audit, or production reliability measurements. Odyssey's contribution at this stage is a documented, integrated platform design and implementation that combines these functions. Establishing educational effectiveness and operational quality requires planned testing and user evaluation beyond the present chapters.

Chapter 3

THEORETICAL AND CONCEPTUAL FRAMEWORK

3.1 Theoretical Framework

3.1.1 Experiential Learning through Guided Simulation

This study draws on the view of simulation-supported learning as guided exploration of a model. Learners can select or change conditions, observe what the model does, and connect outcomes to the concept being studied [6]. Because open-ended exploration can create confusion, Odyssey pairs its labs with lesson material and visible traces. The framework supports the design of interactive practice; it does not predict a measured learning gain for this system.

3.1.2 Cognitive Theory of Multimedia Learning

Mayer's theory explains the potential value of coordinating words and pictures while accounting for limited processing capacity [7]. Odyssey applies this principle through written lesson explanations, diagrams, and visual simulation traces. The design implication is to keep each visual focused, labeled, and connected to the relevant explanation so learners can integrate the representation with prior knowledge.

3.1.3 Iterative Software Development

Odyssey's development is organized around iterative work: identify requirements, design a feature, implement it, inspect its behavior, and refine it. This is consistent with Agile principles of iterative delivery and ongoing review. The repository does not contain evidence of formal sprint records or user feedback for every iteration, so the methodology chapter describes the intended development cycle without claiming a documented cadence or measured stakeholder participation.

3.2 Conceptual Framework

The conceptual framework uses the Input-Process-Output model to show how learner, instructor, system, and study data are transformed into learning and management functions. Inputs include user and classroom data, lesson materials, simulation choices, study content, and progress records. System processes present content, execute simulations, generate study aids, manage classes, and return progress information. Outputs include an expandable learning platform, visible practice traces, study feedback, and administrative workflows.

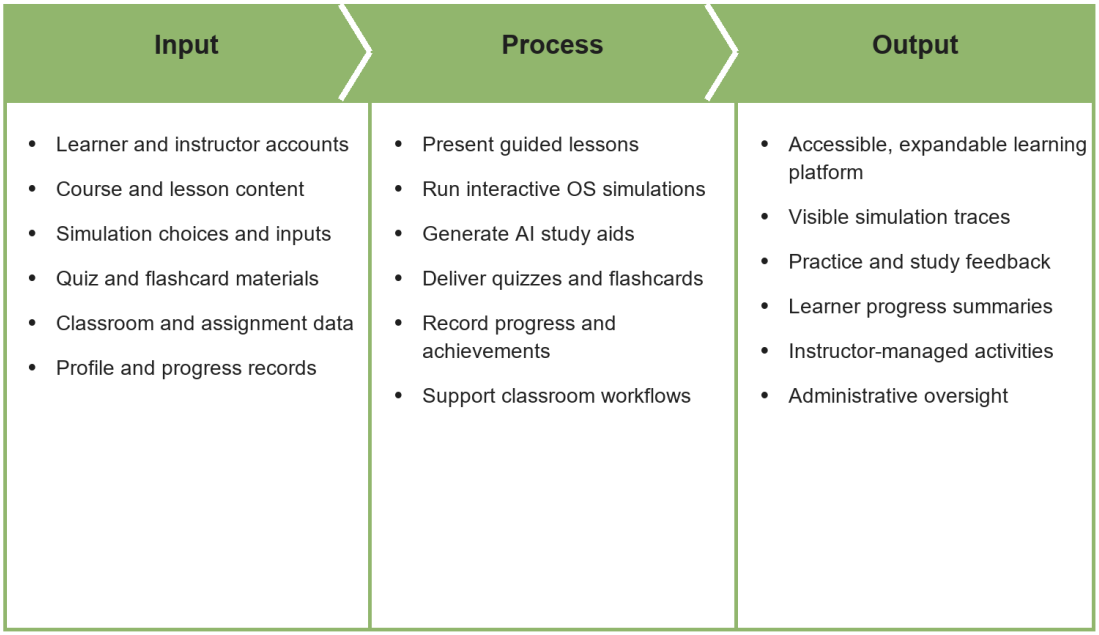


Figure 1. Input-Process-Output Model of Odyssey

As shown in Figure 1, the input stage consists of the learner and instructor accounts, lesson and study materials, simulation choices, classroom information, and available progress records. These inputs represent the people, content, and context that Odyssey uses to provide its learning and classroom functions.

The process stage covers how Odyssey presents lessons, runs interactive operating-systems simulations, creates study aids, delivers quizzes and flashcards, manages classroom activities, and records selected progress. These activities are handled by the client and, where required, the backend API and connected data or AI services.

The output stage includes access to an expandable learning platform, visible simulation traces, study feedback, learner progress summaries, instructor-managed activities, and administrative views. The diagram summarizes the platform's intended information flow; it does not mean every activity is stored on the server. Some selected preferences and learning activity remain in browser storage, and the model does not claim measured learning outcomes.

3.3 System Specification

This section describes Odyssey's documented architecture, deployment arrangement, user roles, representative learner workflows, conceptual data relationships, and development

technology stack. The diagrams summarize system structure and interactions at a high level; they are not deployment-health or performance test results.

3.3.1 System Architecture

The browser presentation layer contains static pages, lessons, simulations, profiles, classroom tools, and the installable PWA shell. FastAPI provides API routes, middleware, authentication and role checks, protected data operations, and AI request handling. Supabase provides authentication and PostgreSQL with Row Level Security. AI requests pass through the backend to the configured provider so credentials are not placed in browser code. Browser storage and the service-worker cache support selected local preferences and static assets.

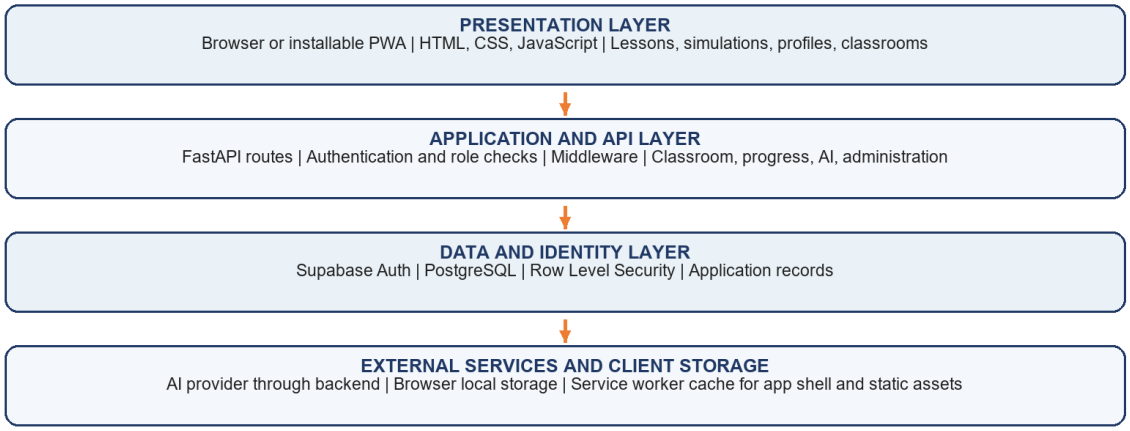


Figure 2. Odyssey System Architecture

3.3.2 Deployment Topology

Repository configuration targets Vercel for static web content and Render for the Python API. Supabase provides hosted authentication and PostgreSQL; configured AI requests use an external AI service. Browsers communicate with these hosted components over HTTPS. This is a configuration-level topology and does not verify active production health, service plans, complete secrets, or network policy.

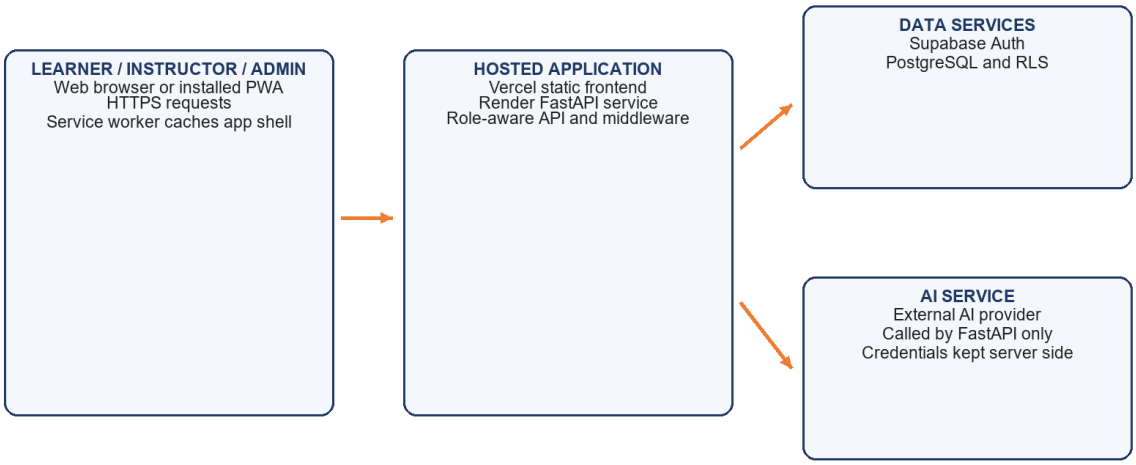


Figure 3. Odyssey Deployment Topology

3.3.3 Use Case Model

The use case model identifies the main human roles visible in the application and groups their supported activities. Learners study modules, run simulations, use study aids, and review progress. Instructors manage classrooms and quizzes. Administrators oversee supported account, role, application, classroom, statistics, and audit functions. External services are technical dependencies rather than human actors.

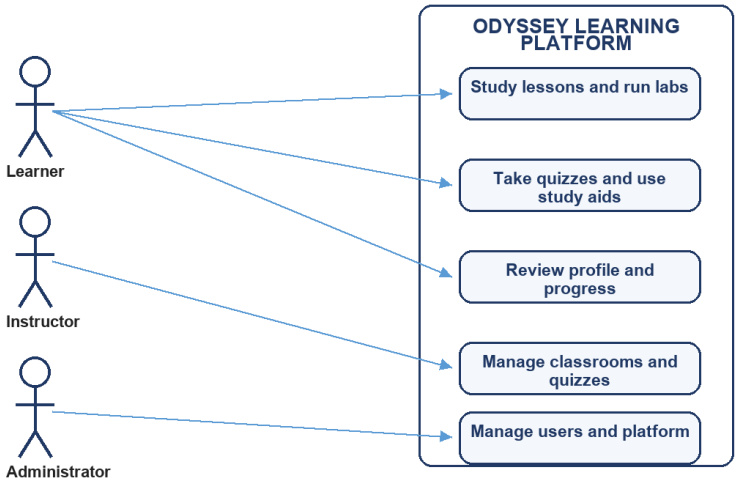


Figure 4. Main Use Case Diagram for Odyssey

3.3.4 Learner Study Activity

The study activity begins when a learner selects an Explore Lessons topic. After reading, the learner may practice using a simulation or quiz, review an explanation, and mark the topic

complete. Whether a progress update reaches the server depends on the specific workflow and network availability; some activity is browser-local.

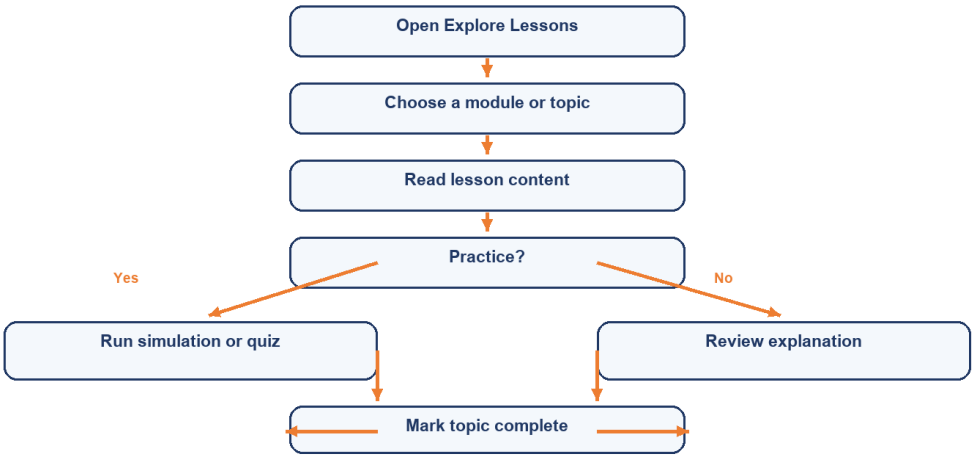


Figure 5. Learner Study Activity Diagram

3.3.5 Protected Progress Request Sequence

Figure 6 shows a representative authenticated progress request. The learner action is sent from the browser to FastAPI, which validates the access token and request before using the configured Supabase client. A successful response returns to the browser for feedback. This is not a claim that every topic click creates a network request.

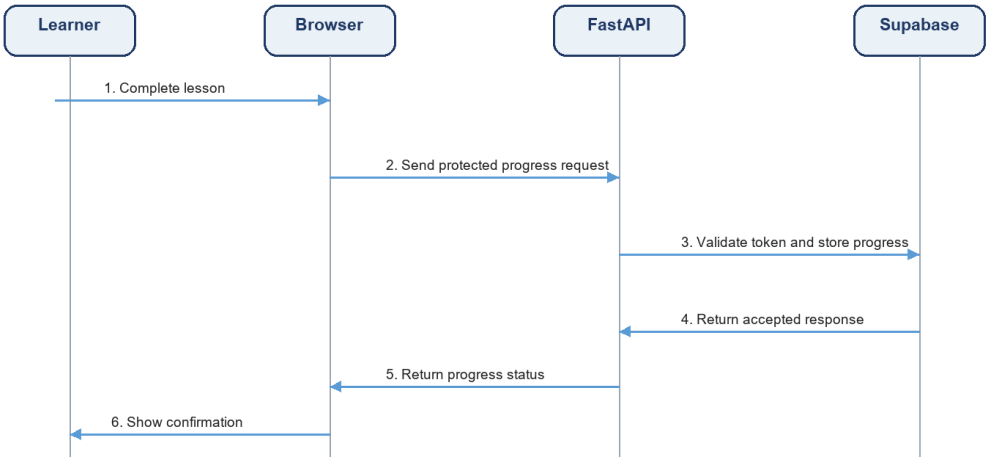


Figure 6. Lesson Completion Request Sequence Diagram

3.3.6 Conceptual Data Model

The conceptual data model summarizes major account and classroom relationships described by the schema and application routes. The authenticated identity is represented by

Supabase Auth and linked to a public profile using a shared UUID. Classroom ownership and membership connect users with classroom workspaces. Modules and quizzes are managed through classroom workflows. Some activity remains local to the browser, so the model is not an exhaustive database diagram.

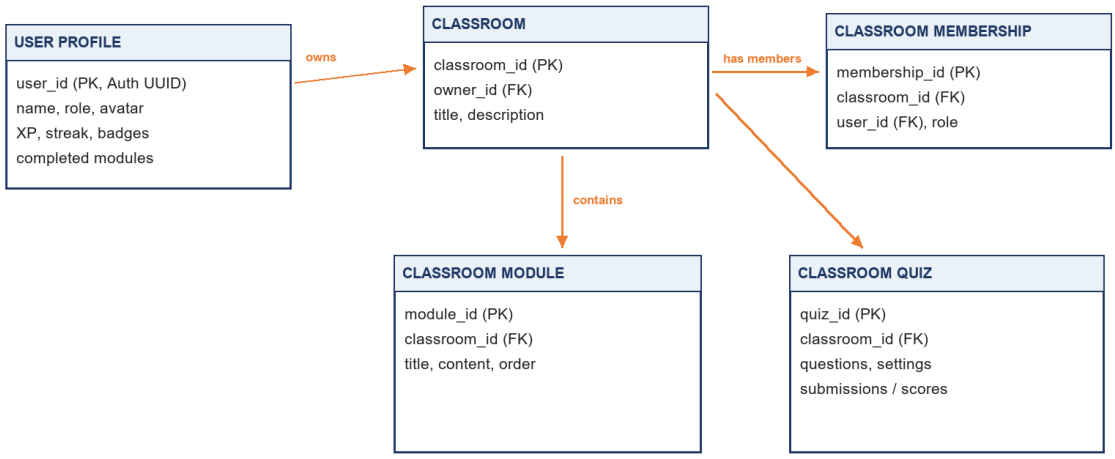


Figure 7. Conceptual Entity Relationship Diagram

The figure is a high-level model rather than a migration-generated ERD. It omits implementation-specific fields and constraints not necessary to explain the study, and it does not imply that each conceptual relationship is enforced by a single database constraint.

3.3.7 Development Environment and Tools

Table 1. Development Technology Stack

Component	Technology	Role in the system
Web client	HTML, CSS, JavaScript	Lessons, simulations, profiles, classrooms, study tools, and PWA behavior.
Backend API	Python 3.10 or newer, FastAPI, Uvicorn	API routing, validation, role-aware operations, AI request mediation.
Authentication and data	Supabase Auth, PostgreSQL, Row Level Security	Identity, relational data, and database access policies.
AI study support	Configured external AI provider via backend	Chatbot responses and quiz or flashcard generation.
Deployment targets	Vercel, Render, Supabase	Static frontend, Python API, identity and database services.
Browser capabilities	Web manifest, service worker, local storage	Installable app shell, static caching, and selected local learning state.

The browser client provides the learner, instructor, and administrator interfaces. FastAPI mediates protected system operations and external AI requests, while Supabase supplies identity and database services. Vercel and Render are the configured deployment targets described by the repository. The table records the documented stack; it does not verify that every service is currently deployed or configured.

Chapter 4

METHODOLOGY

4.1 Research and Development Approach

This work follows a system analysis and design approach focused on documenting the existing Odyssey implementation. Requirements, architecture, roles, and workflows are derived from the supplied Odyssey documentation and repository source files. The approach identifies observable functions and technical boundaries; it does not report participant recruitment, survey results, or controlled educational outcomes because those data were not provided.

The development process is described as iterative software development. Requirements and models are refined as implementation details become available, then checked against the client, backend, database, and deployment configuration. Any future study of user acceptance or learning effectiveness should define participants, instruments, procedures, consent, data handling, and analysis separately before collecting results.

4.2 Agile Development Method

The Agile cycle used to organize Odyssey's feature work is summarized in Figure 8. Planning identifies a feature and its acceptance needs; design describes the user workflow and technical boundary; implementation updates client, API, or data components; testing checks the change; and review informs the next refinement. The cycle can be repeated for lesson, simulation, progress, classroom, administration, or AI functions.

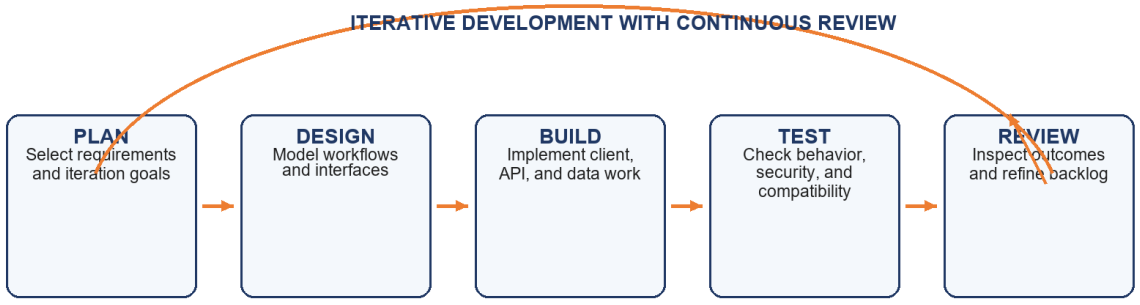


Figure 8. Agile Software Development Cycle

Agile is suitable for a platform whose modules and service integrations can be developed in increments. This documentation does not assert specific iteration dates, sprint lengths, team ceremonies, or stakeholder review records because these were not included in the source material.

4.3 System Requirements

4.3.1 Functional Requirements

The following functional requirements summarize behavior visible in the supplied documentation and repository. Role and persistence boundaries are included where they affect access or system behavior.

Table 2. Functional Requirements

ID	Functional requirement
FR-01	The system shall support account registration, sign-in, sign-out, and password-reset requests through its authentication service.
FR-02	The system shall allow an authenticated user to retrieve and update that user's profile.
FR-03	The system shall validate Supabase access tokens and apply role checks to protected API operations where required.
FR-04	The system shall display available learning modules and topic content.
FR-05	The system shall provide interactive educational simulations for supported operating-systems concepts, including scheduling, memory, processes, threads, file systems, deadlocks, disks, system calls, and boot topics.
FR-06	The system shall allow learners to select or enter scenario inputs and inspect simulation traces or results.
FR-07	The system shall provide quiz, flashcard, and PDF study workflows in the web client.

ID	Functional requirement
FR-08	The system shall provide learner progress summaries and module-completion workflows through supported progress operations.
FR-09	The system shall support XP, streak, badge, profile, statistics, and leaderboard workflows where implemented.
FR-10	The system shall retain selected preferences and local learning activity in browser storage where implemented.
FR-11	The system shall provide a general-purpose AI chatbot and AI workflows that generate quizzes and flashcards from supported materials.
FR-12	The system shall keep AI provider credentials in backend configuration and return generated study content or chatbot responses to the client.
FR-13	The system shall allow authorized instructors to create and manage classrooms and memberships.
FR-14	The system shall allow instructors to manage classroom modules and quizzes, including questions and submissions.
FR-15	The system shall allow authorized administrators to manage supported user records and roles, review instructor applications, inspect statistics and audit entries, and manage classrooms.

4.3.2 Non-Functional Requirements

Table 3. Non-Functional Requirements

ID	Non-functional requirement
NFR-01 Security	Protected API routes shall validate identity and roles; database Row Level Security shall restrict data access.
NFR-02 Transport	Hosted deployment shall use HTTPS, configured CORS allow-lists, security headers, and request-size limits.
NFR-03 Abuse resistance	Configured global, authentication, and API rate limits shall be applied; multi-instance deployments require shared rate-limit storage for consistent enforcement.
NFR-04 Input handling	API inputs shall be validated and sanitized, and client errors shall not expose internal exceptions.
NFR-05 Usability	Lessons and simulations shall present understandable controls and feedback on desktop and mobile layouts.
NFR-06 Availability	The client and API depend on configured hosting and Supabase services; no availability target is documented.
NFR-07 Maintainability	Client pages, API routers, middleware, and service integrations shall remain modular.
NFR-08 Privacy	Account, progress, classroom, and administration data shall be protected through access policies and secure handling of secrets.
NFR-09 Compatibility	The system shall support modern browsers and PWA installation where available; cached static files do not provide offline access to hosted services.
NFR-10 Performance	No response-time or concurrency target is documented; a target should be set after measuring an agreed workload.

4.4 Feasibility Analysis

4.4.1 Technical Feasibility

The repository contains a web client built with HTML, CSS, and JavaScript; a Python FastAPI service; Supabase Auth and PostgreSQL integrations; and route groups for authentication, profiles, progress, modules, statistics, administration, classrooms, AI, applications, and health. Middleware includes request identifiers, security headers, gzip, CORS, rate limiting, sanitization, and logging. The manifest and service worker support installability and caching of selected static files. Repository configuration describes Vercel and Render deployment targets [1]-[5].

Deployment requires valid Supabase settings, database policies and migrations, hosting environment variables, allowed origins, external AI credentials, and network access to hosted services. In-memory rate limiting is not shared across multiple API instances, and browser-local activity may not synchronize across devices. These constraints are technically addressable but require configuration and validation in the target environment.

4.4.2 Organizational Feasibility

The implementation uses common web and database technologies and separates the browser client from API and data responsibilities. Its route and page organization can support contributors with frontend, Python API, database, and deployment skills. Classroom workflows can support instruction when an adopting organization assigns roles and operating responsibilities.

The project records do not identify a sponsoring institution, deployment owner, support team, training plan, or approved operating policy. An adopter should assign responsibility for account administration, classroom setup, content review, privacy requests, incident response, and deployment secrets. Organizational feasibility therefore depends on local ownership and operational readiness.

4.4.3 Economic Feasibility

Table 4. Economic Feasibility Assessment

Category	Evidence-based assessment
Development effort	Source code is available; historical labor hours and labor rates are not recorded.

Category	Evidence-based assessment
Hosting and database	Configuration targets Vercel, Render, and Supabase. Actual plans, usage, paid tiers, and invoices were not provided.
AI service	AI features depend on an external provider; usage volume, quotas, and charges are undocumented.
Maintenance	Ongoing work includes updates, dependency maintenance, content review, security work, and support; no annual estimate is documented.
Potential benefits	Centralized lessons, browser simulations, classroom quizzes, and progress workflows may reduce reliance on separate tools. Benefits have not been measured.
ROI and payback	Cannot be calculated from available evidence. A calculation needs baseline costs and time, implementation and operating costs, measured savings or valued benefits, and a defined evaluation period.

The project is economically plausible as a software product because it uses a static web client, common open-source components, and managed cloud services. Actual economic feasibility remains conditional on local usage, hosting and AI charges, labor, support, and measured value. The source records do not justify a numerical return-on-investment estimate.

4.5 Verification and Evaluation Plan

Verification should confirm that each requirement is observable in the implemented system and that role checks, validation, database policies, and network-dependent workflows behave as intended. The following test groups provide a plan for a later verification stage; they are not reported results.

Table 5. Planned Verification

Evaluation area	Planned verification
Functional verification	Exercise registration, sign-in, reset, profiles, lesson display, simulator inputs and traces, quizzes, flashcards, PDF study, progress, classroom operations, admin workflows, and AI requests.
Authorization and privacy review	Check token validation, role boundaries, Row Level Security policies, secret placement, input sanitization, CORS, security headers, and rate limiting.
Compatibility and usability review	Inspect responsive behavior, keyboard operation, readable labels, feedback, and PWA installation on supported browsers and devices.
Service and network checks	Verify configured deployment, health routes, database connectivity, AI-provider handling, offline app-shell caching, and graceful failure when live services are unavailable.
User evaluation	After approval and participant procedures are defined, collect learner and instructor feedback using agreed usability and learning measures. Report sample sizes, instruments, conditions, and limitations.

Evaluation area	Planned verification
Operational measurement	Measure response times, concurrency, hosting and AI usage, support effort, and any agreed baseline before making reliability or cost claims.

Results, discussion of measured findings, user acceptance results, and conclusions are outside the scope of this document and will be prepared only after the required implementation evidence, evaluation records, and figures are available.

Chapter 5

RESULTS AND DISCUSSION

5.1 Learner Experience and Study Tools

The supplied interface screenshots show an Odyssey home screen with learner currency and streak indicators, a study upload option, and shortcuts to lessons and rankings. The learning-progress screen presents a four-module route, completion status, topic count, and a review action. These are the values and controls visible in the supplied captures; they are not a measurement of overall learner participation or achievement.

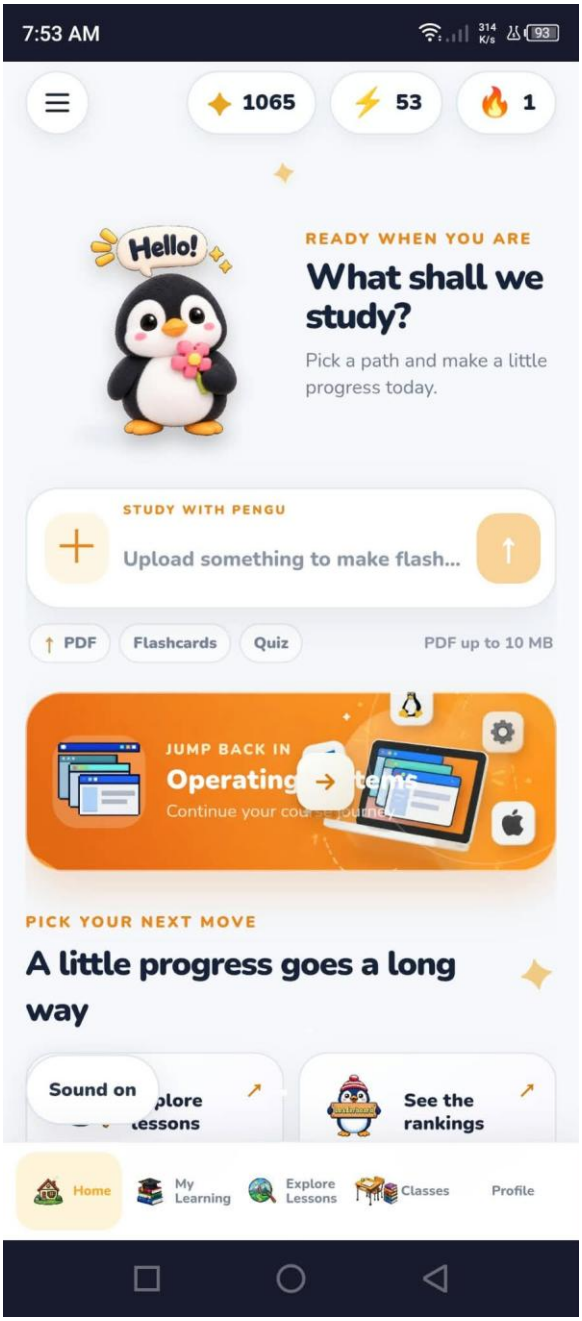


Figure 9. Odyssey home screen and study options

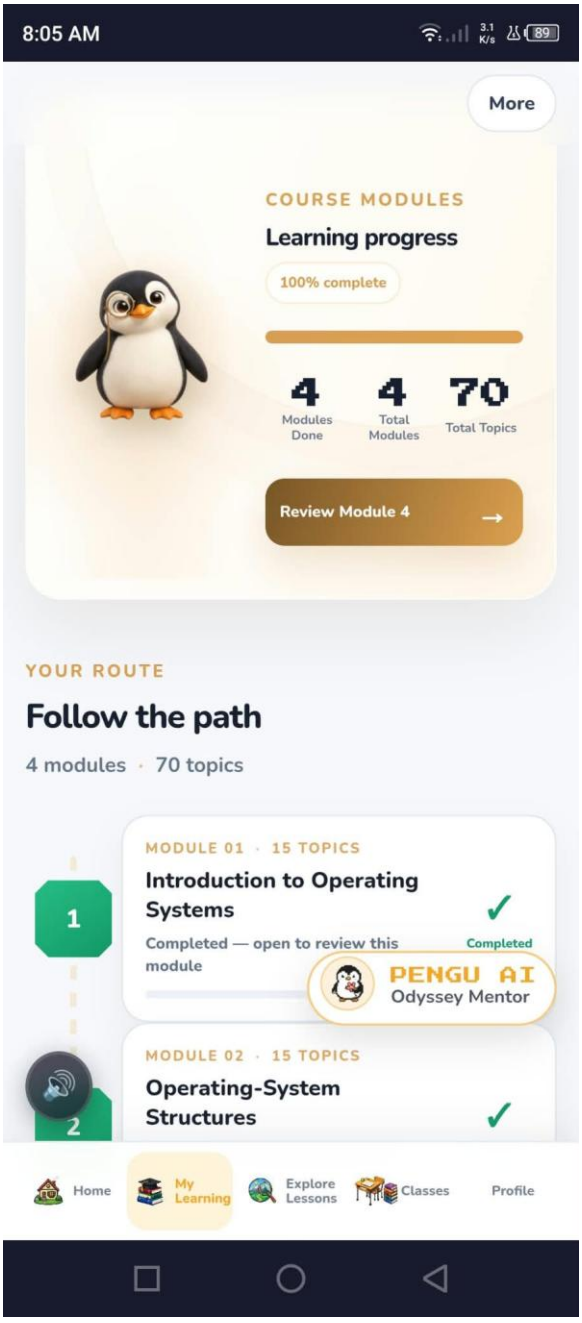


Figure 10. Module route and learning progress

5.2 Pengu Study Aids and Feedback

The screenshots document multiple study modes tied to learner-provided material. The PDF study screen identifies Module 5 as its source and offers memorization, challenge, and question-and-answer options. The flashcard screens show a prompt followed by its revealed explanation. The Quick Check screen shows a multiple-choice question and explanatory feedback. These images illustrate interface states and the displayed example content; they do not establish quiz accuracy across a question bank or improvement in retention.

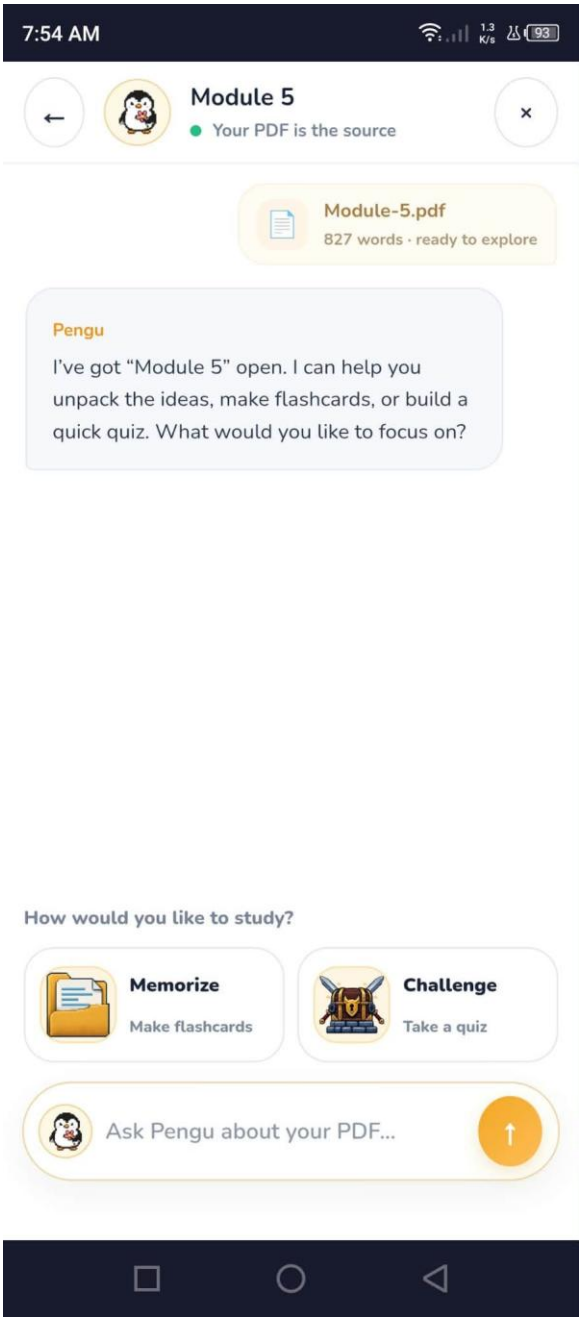


Figure 11. PDF source and Pengu study options

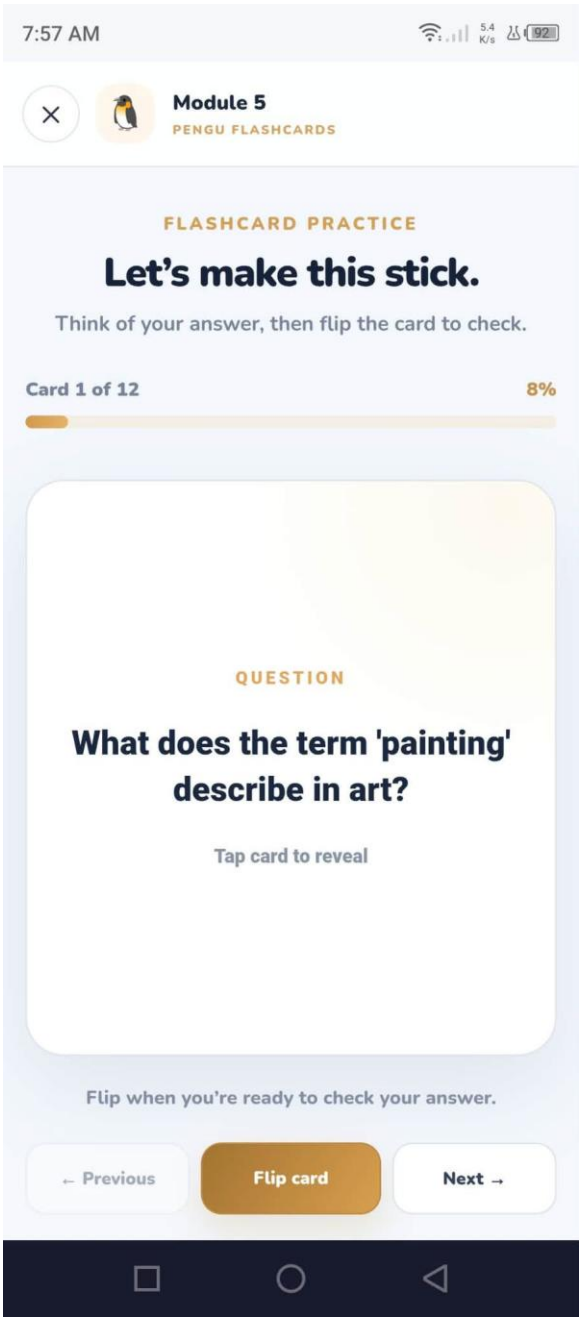


Figure 12. Flashcard question state

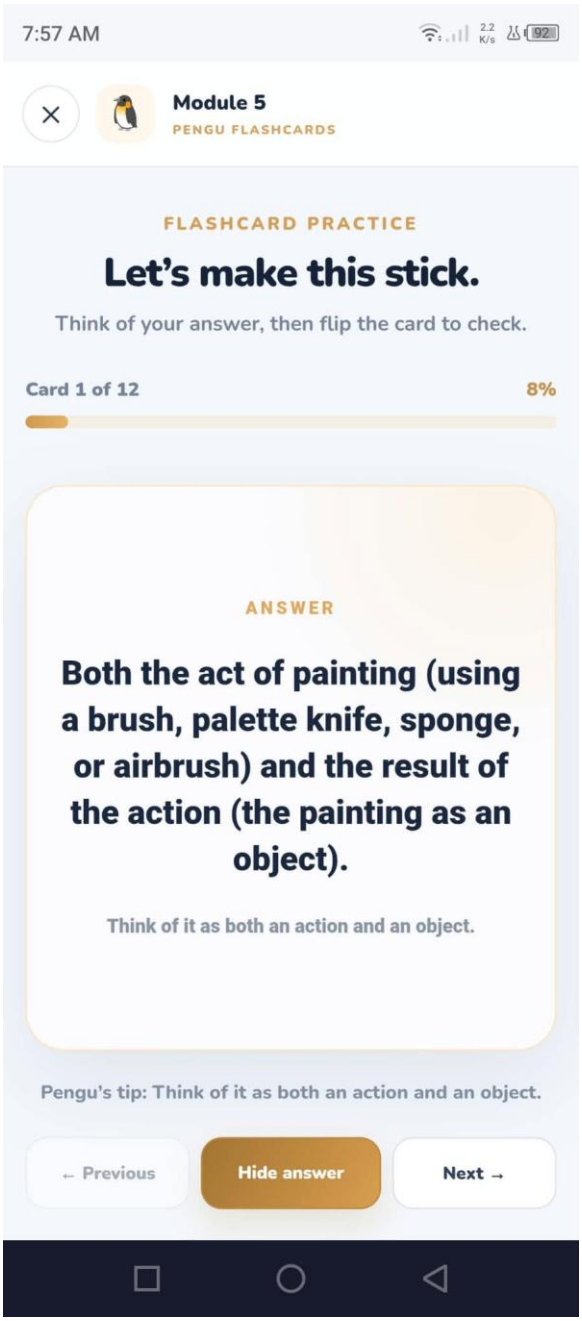


Figure 13. Flashcard answer state

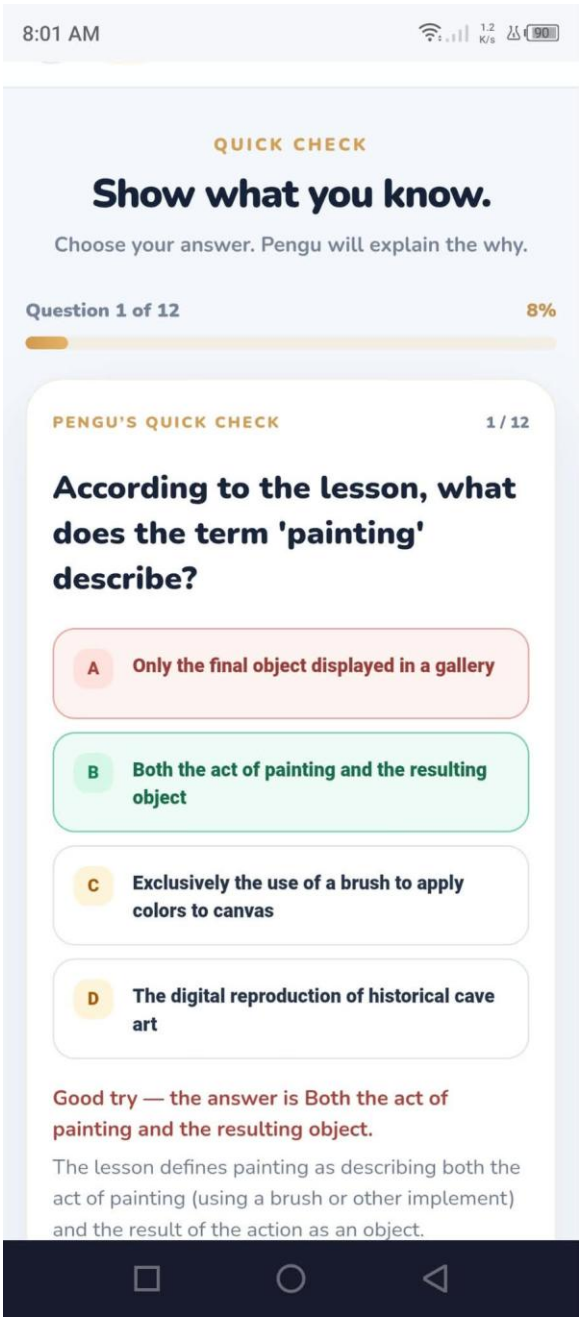


Figure 14. Quick Check question and feedback

5.3 Interactive Operating-Systems Practice

The System Call Tracer screenshot presents a program-selection control, execution and clear actions, a step-by-step option, and a next-step control. It documents the lab interface for exploring transitions between user and kernel mode. The supplied image does not show a completed execution trace, so no specific system-call output or runtime result is reported here.

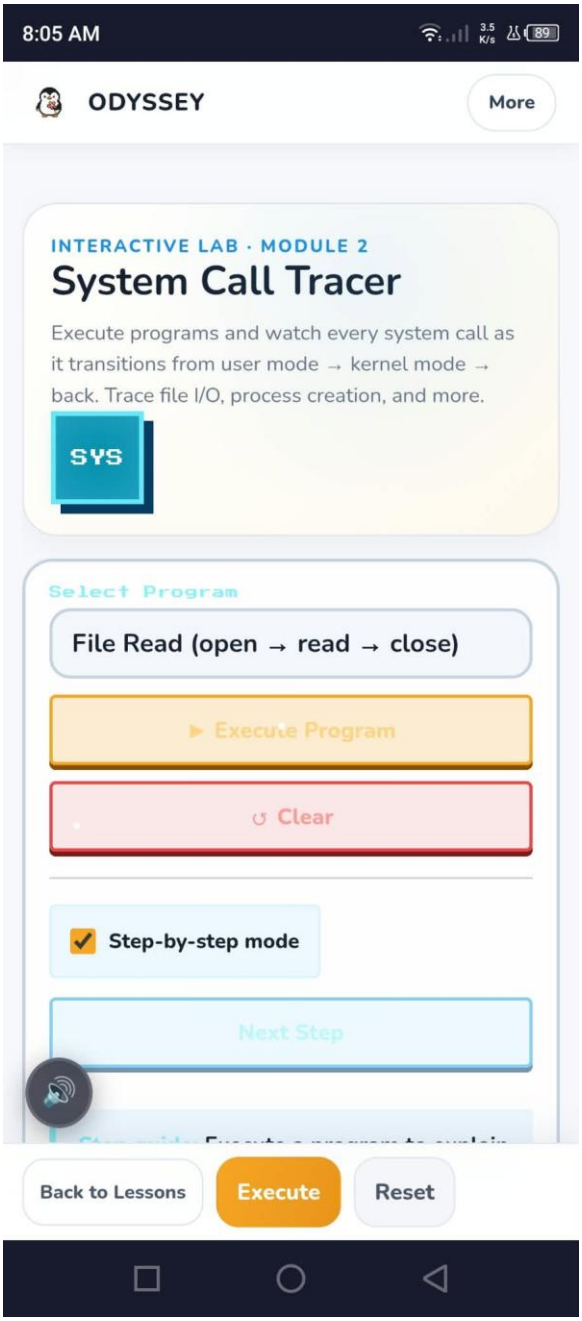


Figure 15. System Call Tracer interface

5.4 Classroom and Instructor Workflows

The classroom screenshots show a join-code field and a listed BSCS 3A classroom, followed by a classroom detail page with module and resource navigation, class options, a progress display, and a quiz-review action. A separate instructor-application screen shows the opening decision step. These captures document selected learner and instructor-facing workflows; they do not show a completed application or verify that all classroom actions succeeded.

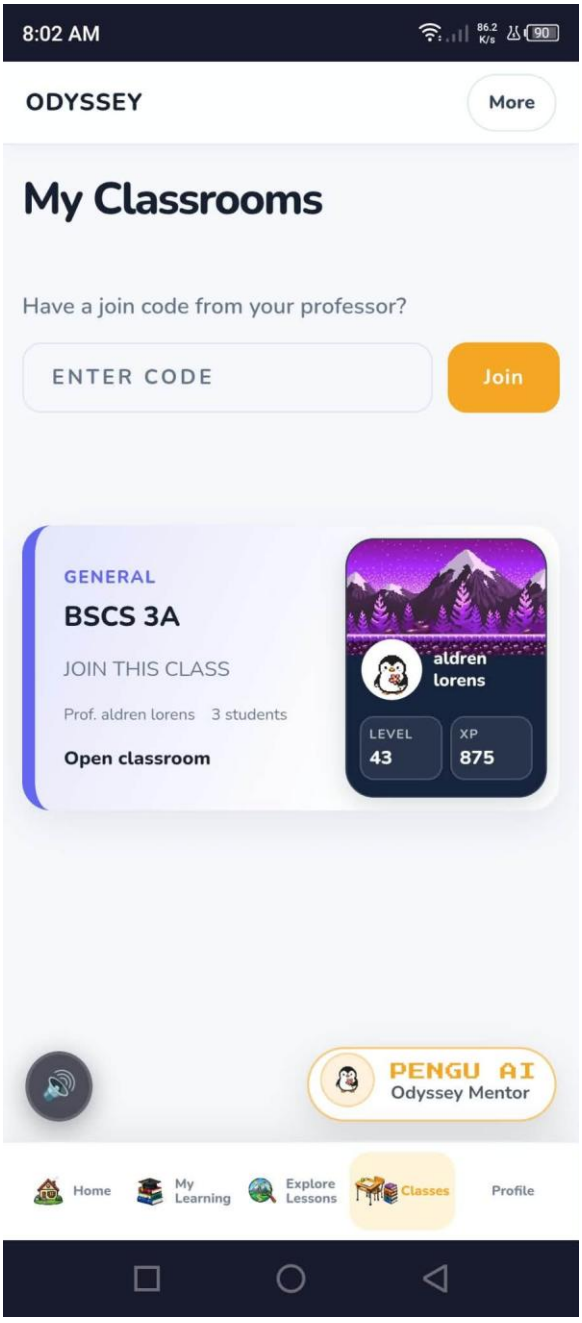


Figure 16. My Classrooms and class-joining interface

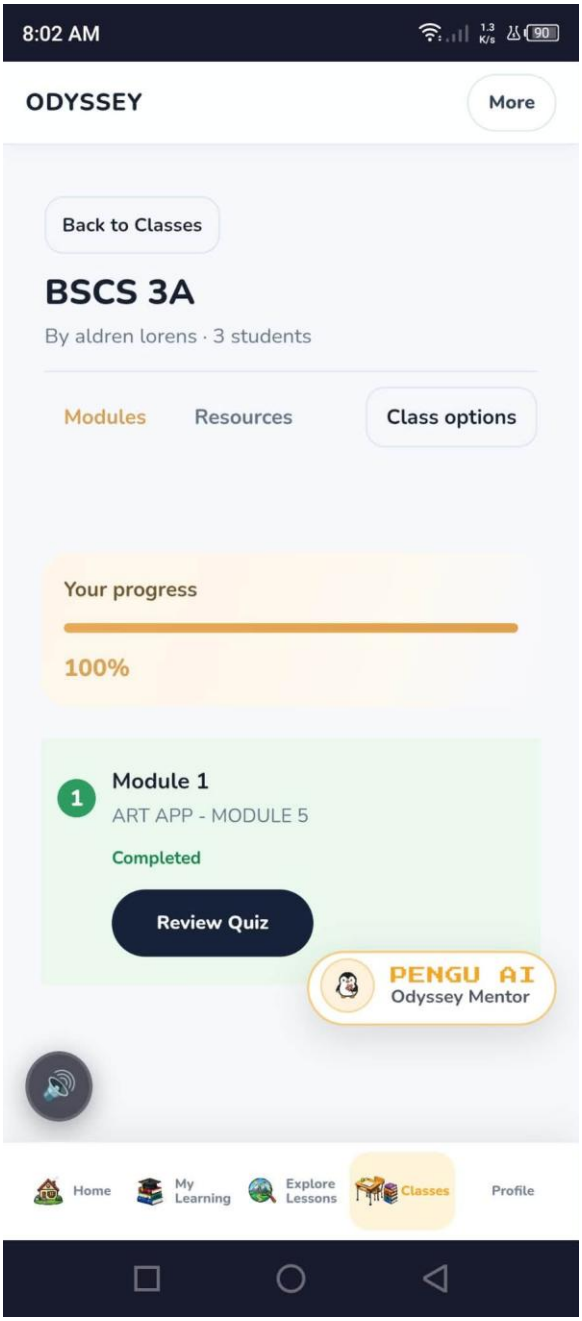


Figure 17. Classroom modules and displayed progress

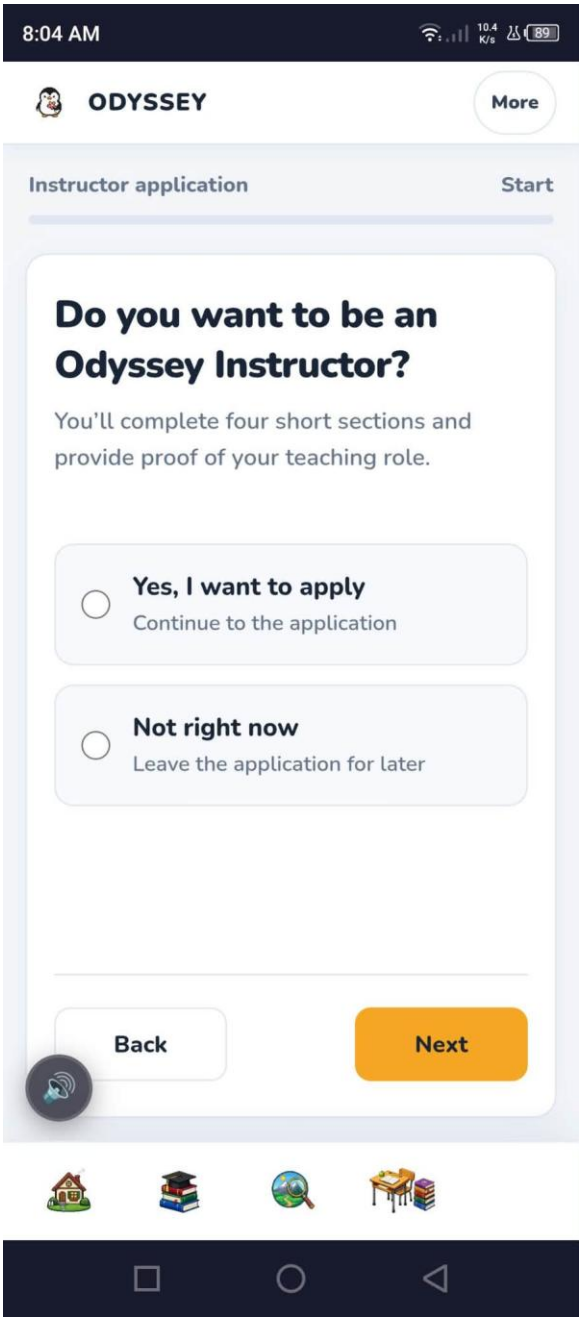


Figure 18. Instructor application opening screen

5.5 Engagement and Administrative Tools

The leaderboard screen ranks learners by XP and includes a badges view. The administrator overview displays account, role, classroom, and average-XP totals, while the users and classrooms screens show management lists. Counts and names shown in these images reflect the captured interface at that moment and should not be treated as current operational totals or independently audited records.

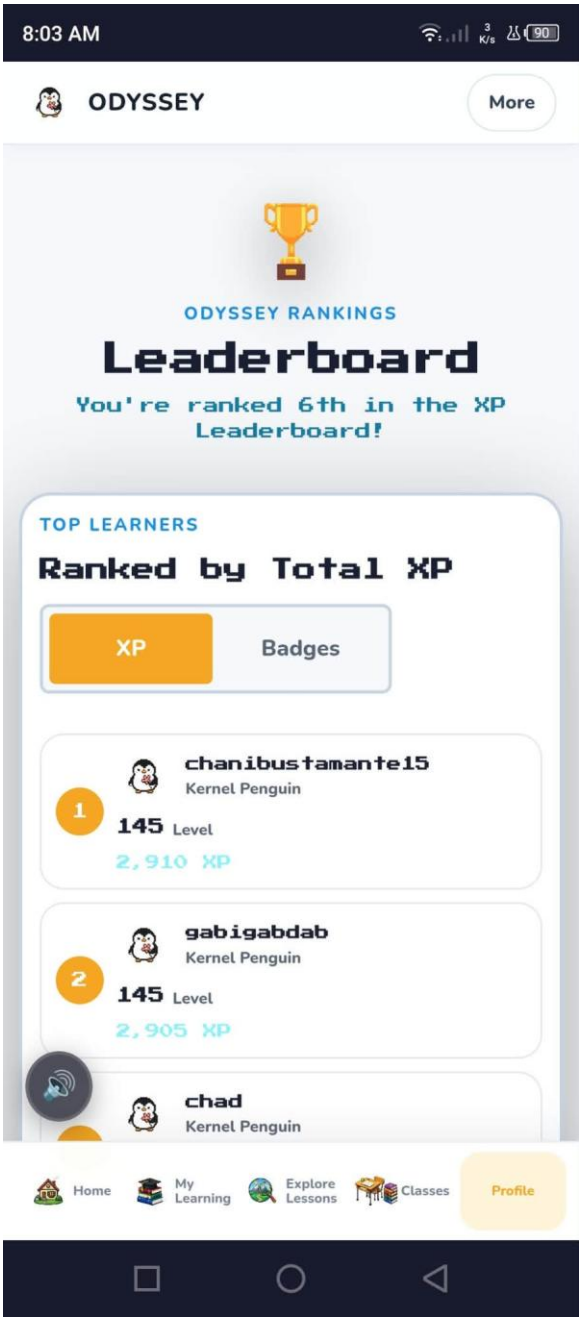


Figure 19. Odyssey learner XP leaderboard

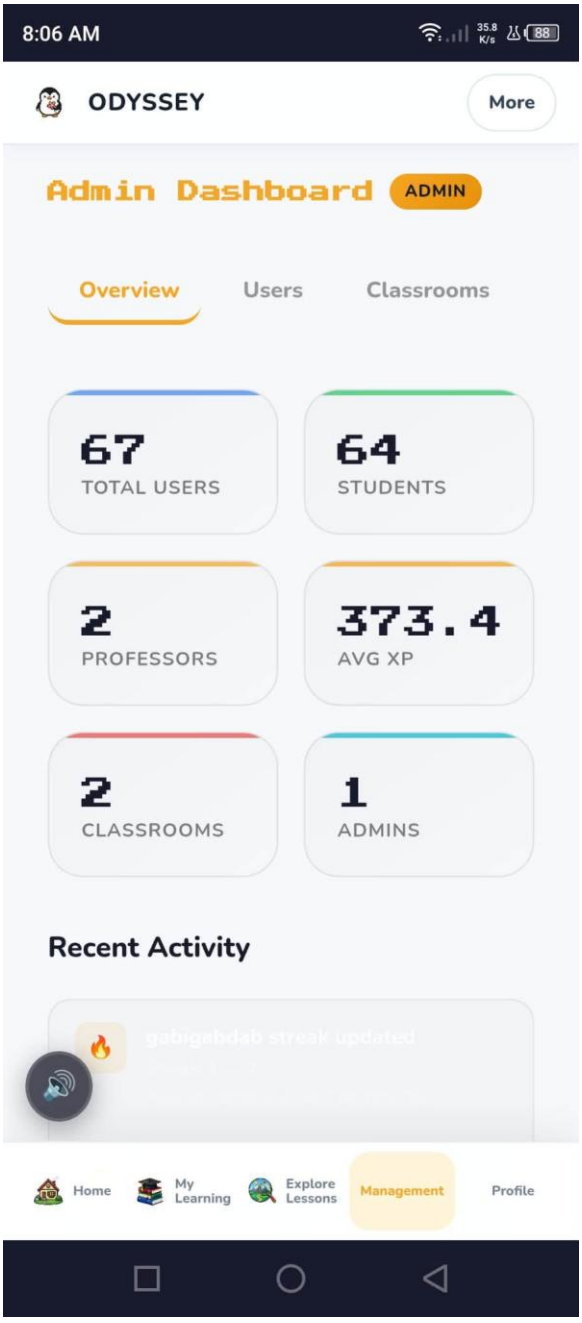


Figure 20. Administrator overview statistics

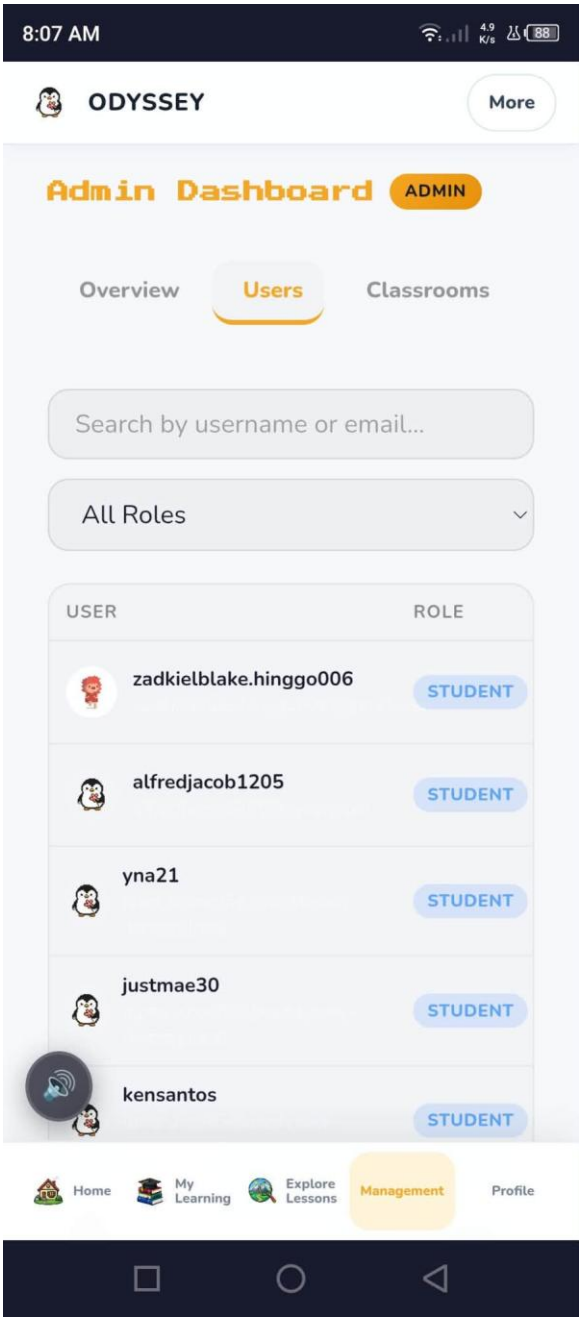


Figure 21. Administrator user management

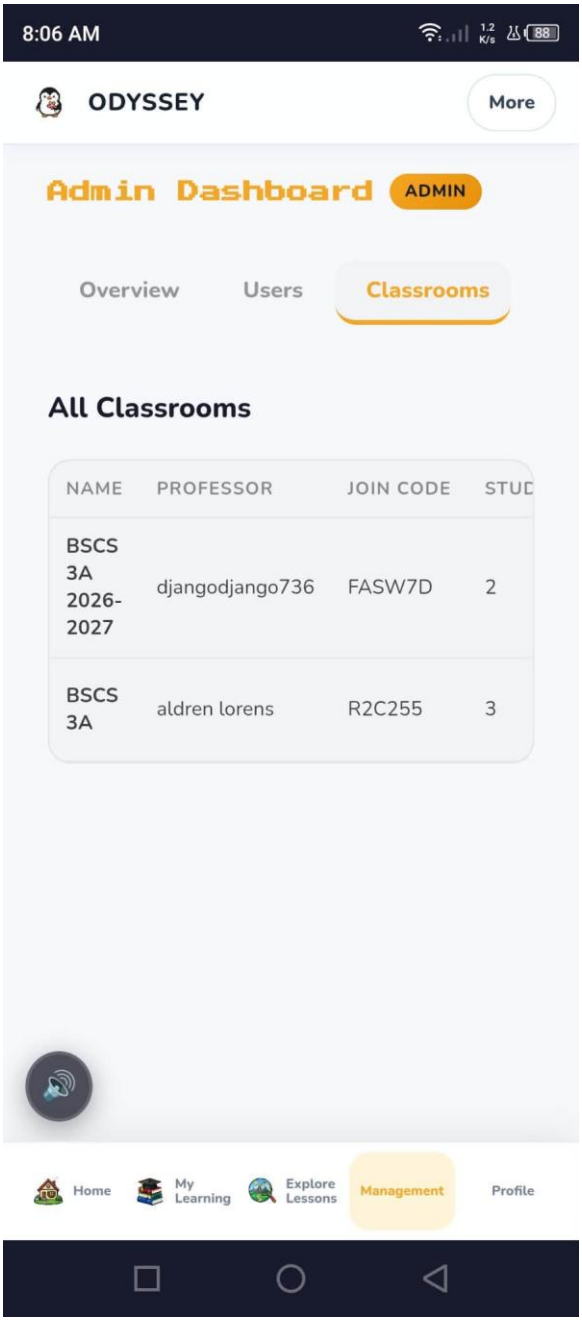


Figure 22. Administrator classroom management

5.6 Discussion and Evidence Limitations

Across the supplied captures, Odyssey presents learner study and progress interfaces, AI-assisted study aids, a system-call lab, classroom tools, a leaderboard, and administration views. This evidence is descriptive and limited to visible screens and example states. It does not verify all controls, security, accessibility, reliability, response time, usability, or learning effectiveness. Those claims require a defined protocol, representative tasks and participants where applicable, collected measures, and documented results, which were not provided.

REFERENCES

[1] Odyssey project repository. Backend README: backend stack, architecture, API routes, security configuration, and setup notes. D:\odyssey-system\os-odyssey\backend\README.md.

[2] Odyssey project repository. FastAPI application factory, middleware, routers, and service initialization. D:\odyssey-system\os-odyssey\backend\app\main.py.

[3] Odyssey project repository. Database schema, Supabase profile table, Row Level Security policies, and profile trigger. D:\odyssey-system\os-odyssey\database\schema.sql.

[4] Odyssey project repository. Frontend HTML, CSS, and JavaScript for lessons, simulations, profiles, classrooms, and AI study workflows. D:\odyssey-system\os-odyssey\frontend\.

[5] Odyssey project repository. Web app manifest, service worker, and deployment configuration. D:\odyssey-system\os-odyssey\manifest.json; D:\odyssey-system\os-odyssey\sw.js; D:\odyssey-system\vercel.json; D:\odyssey-system\render.yaml.

[6] T. de Jong and W. R. van Joolingen. 1998. Scientific discovery learning with computer simulations of conceptual domains. Review of Educational Research 68, 2 (1998), 179-202. https://doi.org/10.3102/00346543068002179.

[7] R. E. Mayer. 2009. Multimedia Learning, 2nd ed. Cambridge University Press. https://doi.org/10.1017/CBO9780511811678.

[8] G. Kestin, K. Miller, A. Klales, T. Milbourne, and G. Ponti. 2025. AI tutoring outperforms in-class active learning: an RCT introducing a novel research-based design in an authentic educational setting. Scientific Reports 15 (2025), 17480. https://doi.org/10.1038/s41598-025-97652-6.

[9] M. Sailer and L. Homner. 2020. The gamification of learning: a meta-analysis. Educational Psychology Review 32 (2020), 77-112. https://doi.org/10.1007/s10648-019-09498-w.